

COMPUTER ENGINEERING

©Copyright: 2026 by james mealy

CPE 439 Experiment 4

Implementing LED Blinking Operations with FreeRTOS Software Timers on the STM32 Development Board: Event Groups

Learning Objectives

- To become familiar with FreeRTOS Event Groups
- To add to your ever growing inventory of simple ways to blink LEDs on a massively functional development board

Introduction and Overview

Embedded systems are typically modeled as systems that have inputs that respond to outputs under program control. Real-time embedded systems take this notion a step further and include a time factor in the mix. The use of an RTOS provides software tools that help make your system a true real-time system but requires that you model the system as a set of tasks that communicate with each other and the outside world. The notion of Event Groups provides yet another synchronization tool for communication purposes but also provides a way to implement sequence restrictions on the running of the various system tasks.

Event groups can act like semaphores with their scheduling abilities, but event groups offer added functionality not offered by semaphores in some situations. When you use semaphores, you can only effectively signal that one event has happened. And though you can schedule this event to multiple tasks that may be waiting on this event, only one of any ready events can take the semaphores. So one of the advantages of event groups is the notion of broadcasting a single event (or multiple events) to multiple tasks in roughly a simultaneous way.

The primary purpose of event groups is to provide a mechanism that allows multiple events to wake multiple tasks waiting on that particular set of events. While the same functionality could roughly be done with other RTOS objects such as semaphores, the event group mechanism is many advantages such as simplicity and less resource utilization.

The idea behind event groups is an event group register, which is a FreeRTOS object. Each bit in the event group register can be used to signal the occurrence of event by setting a bit in the event register; we refer to those bits as event flags. This register allows tasks to block waiting for any set of events in the event register. When a task determines that a set of events has happened, then that task executes some appropriate operation.

Programming Assignment:

Write a program that does the following:

- Blinks an LED at a reasonable frequency.

Constraints:

- Use four different tasks in your implementation.

- One task should set a bit in the event group after a 300ms delay, then signal the second to unblock the second task.
- The second task should then set another event bit before it sends a signal to unblock the third task.
- The third task waits to detect a button press before it sets a third event flag in the event group.
- When the three event flags have been set from the three tasks, a fourth task unblocks and toggles the LED.
- When the directions say signaling, you can signal in any way of your choosing.

Questions:

1. List the two advantages that event groups have over using semaphores for inter-process signaling.
2. Although event groups can handle multiple events, list the two main advantages that using event groups has over using a single semaphore.
3. Briefly describe the two sets of event group functionality associated with the FreeRTOS API **xEventGroupSyncFromISR()**.
4. Briefly describe how event groups can be configured to use either OR or AND functions when waiting for event flags to be set in the event register.
5. Briefly describe how tasks waiting on certain event groups specify which event groups they are waiting for.
6. Briefly describe how tasks can choose to handle what happened to event flags after the task awakens to some set of event flags in the event register being set.

Deliverables:

- ~~1) An in-class demo or video of the working project.~~
- 2) The questions and answers to the questions in the question section.
- 3) A brief but complete description of how you solved this problem.
- 4) A copy of the code you wrote for this experiment (try not to print all the generated code).